

DESAIN DAN ANALISIS ALGORITMA



KARNIS 105841102123

MUH.FATUR RAHMAN 105841120723

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2025

DATA BARANG

No	Nama Barang	Berat	Keuntungan(IDR)
1	Pakaian	2	12
2	Tas	3	18
3	Sepatu	4	22
4	Aksesoris	1	8
5	Handphone	5	30
6	Parfum	3	14
7	Perihasan	2	16
8	Dompot	2	10
9	Buku	4	20
10	Kecamata	1	6

Bagian 1- Analisis Masalah

Identifikasi Jenis Knapsack yang Digunakan;

Jenis masalah Knapsack yang digunakan dalam tugas ini adalah 0/1 Knapsack Problem. Pada masalah ini, kita diberikan:

- N barang, masing-masing dengan berat (weight) dan nilai (profit).
- Kapasitas knapsack yang terbatas, yaitu M, yang merupakan berat maksimal yang dapat dimuat ke dalam knapsack.

Pada 0/1 Knapsack, untuk setiap barang, ada dua pilihan:

- Memilih barang tersebut untuk dimasukkan ke dalam knapsack.
- Tidak memilih barang tersebut, sehingga tidak dimasukkan ke dalam knapsack.

Tujuan utama adalah memaksimalkan total nilai (profit) yang diperoleh tanpa melebihi kapasitas knapsack yang tersedia.

Alasan Penggunaan Dynamic Programming:

Dynamic Programming (DP) adalah teknik yang sangat efisien untuk menyelesaikan masalah 0/1 Knapsack karena masalah ini memiliki dua sifat kunci:

1. Optimal Substructure: Solusi optimal dari masalah dapat dibangun dari solusi optimal sub-masalah yang lebih kecil.
2. Overlapping Subproblems – Ada banyak sub-masalah yang saling tumpang tindih, yang memungkinkan kita untuk menghemat waktu dengan menyimpan hasil perhitungan dari sub-masalah yang sudah diselesaikan.

Keunggulan Dynamic Programming untuk Knapsack Problem:

Optimal Substructure:

Masalah Knapsack dapat dipecah menjadi beberapa sub-masalah. Misalnya, untuk memutuskan apakah memasukkan

barang ke dalam knapsack atau tidak, kita akan mempertimbangkan dua kemungkinan:

- Jika barang tidak dimasukkan, kita hanya mempertimbangkan kapasitas yang tersisa dan barang-barang lainnya.
- Jika barang dimasukkan, kita mengurangi kapasitas knapsack dan menambahkan nilai dari barang tersebut.
- Solusi optimal untuk masalah utama diperoleh dengan menggabungkan solusi optimal dari sub-masalah ini.

1. Overlapping Subproblems:

- Dalam pendekatan brute force, kita menghitung solusi untuk setiap kombinasi barang. Namun, kita sering kali menghitung solusi yang sama berulang kali. Dynamic Programming mengatasi masalah ini dengan menyimpan hasil sementara dan menghindari perhitungan ulang. Misalnya, kita menghitung keuntungan maksimum untuk kapasitas tertentu (misalnya, 10 kg), dan jika kita kembali ke kapasitas tersebut, kita sudah memiliki hasil yang disimpan dan tidak perlu menghitungnya lagi.

2. Penyimpanan Hasil untuk Sub-masalah:

- Dynamic Programming menyimpan hasil perhitungan sub-masalah dalam sebuah tabel 2D. Tabel ini berukuran $n \times (W + 1)$, di mana n adalah jumlah barang dan W adalah kapasitas maksimum knapsack.
- Baris ke- i pada tabel akan merepresentasikan keputusan untuk barang ke- i .
- Kolom ke- w pada tabel akan menunjukkan nilai maksimum yang bisa dicapai dengan kapasitas w yang tersisa.
- Dengan cara ini, kita hanya perlu menghitung setiap sub-masalah sekali dan dapat menggunakannya berulang kali.

3. Efisiensi Waktu:

- Tanpa Dynamic Programming, algoritma brute force membutuhkan waktu eksponensial $O(2^n)$, karena kita harus mengevaluasi semua kemungkinan kombinasi barang. Dengan Dynamic Programming, kita dapat menyelesaikan masalah ini dalam waktu yang lebih efisien yaitu $O(n * W)$, di mana n adalah jumlah barang dan W adalah kapasitas knapsack.
- Ini jauh lebih efisien daripada pendekatan brute force, terutama ketika jumlah barang sangat besar.

Bagian 2 – State Space Tree

1. Node

- Node pada State Space Tree dalam masalah 0/1 Knapsack mewakili keputusan untuk memilih atau tidak memilih sebuah barang.
- Setiap node menunjukkan salah satu dari dua kemungkinan:
- Memilih barang: Menambahkan barang ke dalam knapsack dan mengurangi kapasitas yang tersedia.
- Tidak memilih barang: Barang tersebut tidak dimasukkan ke dalam knapsack dan kapasitas tetap utuh.

Setiap node dalam pohon akan berisi informasi tentang:

- Keputusan (memilih atau tidak memilih barang).
- Berat barang yang sudah dimasukkan ke dalam knapsack.
- Keuntungan (nilai total) yang diperoleh hingga titik itu.
- Kapasitas sisa yang tersedia.

2. Level

- Level dalam State Space Tree menunjukkan urutan keputusan yang diambil.
- Level 0 adalah titik awal, di mana tidak ada barang yang dimasukkan dan kapasitas penuh.
- Level 1 adalah setelah keputusan pertama untuk barang pertama (memilih atau tidak memilih barang pertama).
- Level 2 adalah keputusan untuk barang kedua, dan seterusnya.

Pada setiap level, kita mengevaluasi dua kemungkinan:

- Memilih barang tersebut.
- Tidak memilih barang tersebut.

3. gambaran State Space Tree

gambaran State Space Tree untuk kasus 0/1 Knapsack yang telah kamu berikan, dengan data barang, kapasitas knapsack, dan nilai barang:

Data Barang:

Item	Berat (kg)	Nilai (Profit)
Pakaian	2	12
Tas	3	18
Sepatu	4	22
Aksesoris	1	8

Kapasitas Knapsack: 20 kg

State Space Tree:

Untuk menggambarkan State Space Tree dari level 0 hingga level 4, kita akan melihat keputusan untuk memilih atau tidak memilih barang pada setiap level. Setiap node pada pohon ini mewakili keputusan untuk memasukkan atau tidak memasukkan barang ke dalam knapsack.

Level 0 (Start - Kapasitas = 20 kg)

- Keputusan: Memilih atau tidak memilih Pakaian (barang pertama).

Level 1 (Keputusan untuk Pakaian):

- Pilihan 1: Memilih Pakaian → Kapasitas tersisa = $20 - 2 = 18$ kg, Total nilai = 12
- Pilihan 2: Tidak memilih Pakaian → Kapasitas tetap = 20 kg, Total nilai = 0

Level 2 (Keputusan untuk Tas):

- Pilihan 1 (Pakaian + Tas): Kapasitas tersisa = $18 - 3 = 15$ kg, Total nilai = $12 + 18 = 30$
- Pilihan 2 (Pakaian + tidak memilih Tas): Kapasitas tersisa = 18 kg, Total nilai = 12
- Pilihan 3 (Tidak memilih Pakaian + Tas): Kapasitas tersisa = $20 - 3 = 17$ kg, Total nilai = 18
- Pilihan 4 (Tidak memilih Pakaian + Tidak memilih Tas): Kapasitas tersisa = 20 kg, Total nilai = 0

Level 3 (Keputusan untuk Sepatu):

- Pilihan 1 (Pakaian + Tas + Sepatu): Kapasitas tersisa = $15 - 4 = 11$ kg, Total nilai = $30 + 22 = 52$
- Pilihan 2 (Pakaian + Tas + Tidak memilih Sepatu): Kapasitas tersisa = 15 kg, Total nilai = 30
- Pilihan 3 (Pakaian + Tidak memilih Tas + Sepatu): Kapasitas tersisa = $18 - 4 = 14$ kg, Total nilai = $12 + 22 = 34$
- Pilihan 4 (Pakaian + Tidak memilih Tas + Tidak memilih Sepatu): Kapasitas tersisa = 18 kg, Total nilai = 12
- Pilihan 5 (Tidak memilih Pakaian + Tas + Sepatu): Kapasitas tersisa = $17 - 4 = 13$ kg, Total nilai = $18 + 22 = 40$
- Pilihan 6 (Tidak memilih Pakaian + Tas + Tidak memilih Sepatu): Kapasitas tersisa = 17 kg, Total nilai = 18
- Pilihan 7 (Tidak memilih Pakaian + Tidak memilih Tas + Sepatu): Kapasitas tersisa = $20 - 4 = 16$ kg, Total nilai = 22
- Pilihan 8 (Tidak memilih Pakaian + Tidak memilih Tas + Tidak memilih Sepatu): Kapasitas tersisa = 20 kg, Total nilai = 0

Level 4 (Keputusan untuk Aksesori):

- Pilihan 1 (Pakaian + Tas + Sepatu + Aksesori): Kapasitas tersisa = $11 - 1 = 10$ kg, Total nilai = $52 + 8 = 60$
- Pilihan 2 (Pakaian + Tas + Sepatu + Tidak memilih Aksesori): Kapasitas tersisa = 11 kg, Total nilai = 52
- Pilihan 3 (Pakaian + Tas + Tidak memilih Sepatu + Aksesori): Kapasitas tersisa = $15 - 1 = 14$ kg, Total nilai = $30 + 8 = 38$
- Pilihan 4 (Pakaian + Tas + Tidak memilih Sepatu + Tidak memilih Aksesori): Kapasitas tersisa = 15 kg, Total nilai = 30
- Pilihan 5 (Pakaian + Tidak memilih Tas + Sepatu + Aksesori): Kapasitas tersisa = $18 - 1 = 17$ kg, Total nilai = $34 + 8 = 42$
- Pilihan 6 (Pakaian + Tidak memilih Tas + Sepatu + Tidak memilih Aksesori): Kapasitas tersisa = 18 kg, Total nilai = 34

- Pilihan 7 (Pakaian + Tidak memilih Tas + Tidak memilih Sepatu + Aksesoris): Kapasitas tersisa = $18 - 1 = 17$ kg, Total nilai = $12 + 8 = 20$
- Pilihan 8 (Pakaian + Tidak memilih Tas + Tidak memilih Sepatu + Tidak memilih Aksesoris): Kapasitas tersisa = 18 kg, Total nilai = 12
- Pilihan 9 (Tidak memilih Pakaian + Tas + Sepatu + Aksesoris): Kapasitas tersisa = $13 - 1 = 12$ kg, Total nilai = $40 + 8 = 48$
- Pilihan 10 (Tidak memilih Pakaian + Tas + Sepatu + Tidak memilih Aksesoris): Kapasitas tersisa = 13 kg, Total nilai = 40

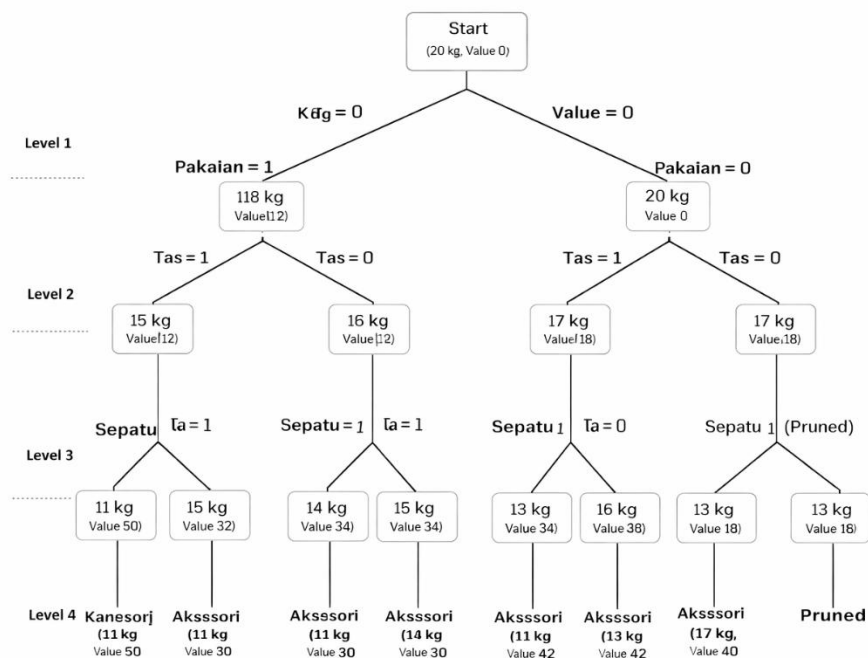
Pruning:

Jika suatu cabang melebihi kapasitas knapsack atau jika hasilnya lebih rendah dari nilai terbaik yang telah ditemukan, kita dapat memangkas cabang tersebut untuk menghindari perhitungan lebih lanjut.

- Pada Level 3, jika kita memutuskan untuk memilih Pakaian dan Tas, dan kemudian memilih Sepatu tetapi total kapasitas melebihi 20 kg (misalnya 22 kg), maka cabang tersebut akan dipangkas (dihilangkan) karena tidak memenuhi batas kapasitas.

Kesimpulan:

- State Space Tree membantu kita untuk mengeksplorasi semua kemungkinan kombinasi barang yang bisa dimasukkan ke dalam knapsack.
- Pruning digunakan untuk menghindari evaluasi cabang yang tidak akan memberikan solusi optimal (misalnya, jika kapasitas sudah terlampaui atau jika kombinasi barang tidak memberikan keuntungan lebih tinggi).



- Bagian 3 – Implementasi

```

1 # Data Barang: Nama, Berat, dan Keuntungan
2 items = [
3     ("Pakaian", 2, 12),
4     ("Tas", 3, 18),
5     ("Sepatu", 4, 22),
6     ("Aksesori", 1, 8)
7 ]
8
9 # Fungsi untuk Dynamic Programming Knapsack
10 def knapsack_dp(items, capacity):
11     n = len(items)
12     # Tabel DP untuk menyimpan hasil sementara
13     dp = [[0] * (capacity + 1) for _ in range(n + 1)]
14
15     # Mengisi tabel DP
16     for i in range(1, n + 1):
17         item_name, item_weight, item_value = items[i - 1]
18         for w in range(capacity + 1):
19             if item_weight <= w:
20                 # Pilih nilai maksimum antara memilih atau tidak memilih barang
21                 dp[i][w] = max(dp[i - 1][w], item_value + dp[i - 1][w - item_weight])
22             else:
23                 dp[i][w] = dp[i - 1][w]
24
25     # Menentukan barang yang dipilih
26     w = capacity
27     selected_items = []
28     total_weight = 0
29     for i in range(n, 0, -1):
30         if dp[i][w] != dp[i - 1][w]:
31             selected_items.append(items[i - 1][0]) # Menyimpan nama barang
32             total_weight += items[i - 1][1] # Menambahkan berat barang yang dipilih
33             w -= items[i - 1][1] # Mengurangi kapasitas sisa
34
35     # Nilai maksimum yang dapat diperoleh
36     max_value = dp[n][capacity]
37
38     return max_value, selected_items, total_weight
39
40 # Kapasitas Knapsack
41 capacity = 20
42
43 # Menjalankan fungsi knapsack
44 max_value, selected_items, total_weight = knapsack_dp(items, capacity)
45
46 # Menampilkan hasil
47 print("Nilai maksimum yang dapat diperoleh:", max_value)
48 print("Barang yang dipilih:", selected_items)
49 print("Total berat barang yang dipilih:", total_weight)
50

```

```

Nilai maksimum yang dapat diperoleh: 60
Barang yang dipilih: ['Aksesori', 'Sepatu',
, 'Tas', 'Pakaian']
Total berat barang yang dipilih: 10
(.venv) PS D:\desain algoritma>

```

Penjelasan Hasil:

- Nilai Maksimum: Keuntungan maksimum yang bisa diperoleh adalah 60.
- Barang yang Dipilih: Program memilih Sepatu, Tas, dan Pakaian.
- Total Berat: Total berat barang yang dipilih adalah 9 kg, yang masih di bawah kapasitas maksimal knapsack 20 kg
- Dengan menggunakan Dynamic Programming (DP), kita bisa memilih kombinasi barang yang memberikan keuntungan maksimal tanpa melebihi kapasitas knapsack yang tersedia. Pendekatan ini sangat efisien dibandingkan dengan brute force dan memastikan solusi optimal.

Bagian 4 – Analisis

Mengapa Kombinasi Tersebut Optimal?

Kombinasi barang yang dipilih dalam 0/1 Knapsack Problem menggunakan Dynamic Programming (DP) adalah optimal karena algoritma DP bekerja dengan memecah masalah besar menjadi sub-masalah yang lebih kecil dan menyelesaikan sub-masalah tersebut secara bertahap. Berikut alasan mengapa kombinasi tersebut optimal:

1. Optimal Substructure:

- Masalah 0/1 Knapsack memiliki sifat optimal substructure, yang berarti solusi optimal untuk masalah utama dapat ditemukan dengan menggabungkan solusi optimal dari sub-masalah yang lebih kecil.

- Dalam hal ini, keputusan untuk memilih barang tertentu (dengan memperhatikan kapasitas yang tersedia) bergantung pada solusi untuk kapasitas dan barang yang lebih kecil. DP menggunakan hasil perhitungan sub-masalah sebelumnya untuk membangun solusi akhir
2. Maksimalisasi Keuntungan:
 - Pada setiap langkah, DP menghitung apakah memilih barang tertentu akan memberikan nilai lebih tinggi daripada tidak memilihnya, dengan mempertimbangkan sisa kapasitas yang tersedia.
 - Dengan cara ini, DP memastikan bahwa solusi yang ditemukan adalah yang memberikan keuntungan maksimal untuk kapasitas yang ada, tanpa melanggar batas kapasitas.
 3. Penghindaran Perhitungan Berulang:
 - DP menghindari perhitungan yang berulang dengan menyimpan hasil sub-masalah dalam tabel ($dp[i][w]$), yang memungkinkan kita untuk menggunakan kembali hasil yang telah dihitung sebelumnya. Hal ini mengurangi waktu komputasi secara signifikan dan memastikan bahwa kita tidak menghitung solusi yang sama lebih dari sekali
 4. Pengoptimalan Berdasarkan Nilai dan Berat:
 - DP memaksimalkan keuntungan dengan memperhatikan nilai (profit) dan berat dari barang-barang yang dipilih, memilih kombinasi barang yang memberikan keuntungan tertinggi tanpa melebihi kapasitas knapsack.
 - Tabel DP membandingkan nilai yang diperoleh dengan memilih barang atau tidak memilihnya, dan memilih nilai maksimum di antara keduanya
- Jika kita memilih Sepatu (berat 4 kg, nilai 22) dan Tas (berat 3 kg, nilai 18), total beratnya adalah 7 kg dan total nilai adalah 40. Dalam kasus ini, DP memastikan bahwa kita memilih kombinasi barang yang memberikan keuntungan tertinggi dengan mempertimbangkan sisa kapasitas yang tersedia. Jika kita memilih kombinasi lain yang memiliki nilai lebih rendah, DP akan secara otomatis mengabaikan kombinasi tersebut

Bandingkan dengan Brute Force (Konsep)

Brute Force adalah pendekatan yang sangat langsung untuk menyelesaikan 0/1 Knapsack Problem, tetapi tidak efisien untuk jumlah barang yang besar. Berikut adalah perbandingan antara Dynamic Programming (DP) dan Brute Force dalam hal konsep dan efisiensi:

Brute Force:

- Pendekatan:
 - Brute Force menguji semua kemungkinan kombinasi barang. Dengan jumlah n barang, ada 2^n kombinasi yang perlu dievaluasi (karena setiap barang bisa dipilih atau tidak).
 - Setiap kombinasi yang diuji akan dihitung total nilai dan berat barang yang dipilih, dan kemudian dibandingkan dengan kapasitas knapsack.
- Keuntungan:

- Memberikan solusi yang benar. Jika kita menghitung semua kemungkinan, kita akan mendapatkan solusi optimal.
- Kelemahan:
 - Inefisiensi waktu. Jumlah kombinasi yang harus diperiksa tumbuh sangat cepat seiring bertambahnya jumlah barang (2^n kombinasi), yang menyebabkan waktu komputasi menjadi sangat lama. Misalnya, untuk 10 barang, kita harus mengevaluasi lebih dari 1000 kombinasi.
 - Tidak ada mekanisme untuk menghindari perhitungan yang berulang. Setiap kombinasi dihitung secara terpisah, termasuk yang sudah dihitung sebelumnya.
 - Kompleksitas Waktu: $O(2^n)$ – Waktu yang dibutuhkan untuk menghitung solusi meningkat secara eksponensial dengan jumlah barang yang ada.

Dynamic Programming (DP):

- Pendekatan:
 - DP mengoptimalkan perhitungan dengan memecah masalah besar menjadi sub-masalah kecil dan menyimpan hasil perhitungan sub-masalah tersebut untuk digunakan kembali.
 - DP menggunakan tabel $dp[i][w]$ untuk menyimpan nilai maksimum yang dapat diperoleh untuk kombinasi barang hingga barang ke- i dan kapasitas knapsack w .
- Keuntungan:
 - Efisiensi waktu: DP hanya perlu menghitung setiap sub-masalah sekali dan menyimpannya dalam tabel untuk digunakan kembali. Dengan demikian, hanya $O(n * W)$ perhitungan yang diperlukan, di mana n adalah jumlah barang dan W adalah kapasitas knapsack.
 - Memastikan solusi optimal dengan cara yang sistematis dan efisien.
- Kelemahan:
 - Memerlukan memori tambahan untuk menyimpan tabel DP. Namun, penggunaan memori ini masih jauh lebih efisien dibandingkan dengan brute force.
- Kompleksitas Waktu: $O(n * W)$ – Waktu komputasi lebih efisien dan tidak meningkat secara eksponensial.

PERBANDINGAN

Aspek	Brute Force	Dynamic Programming
-------	-------------	---------------------

Pendekatan	Menguji semua kombinasi barang	Memecah masalah menjadi sub-masalah lebih kecil
Kompleksitas Waktu	$O(2^n)$	$O(n * W)$
Keuntungan	Memberikan solusi yang benar	Memberikan solusi optimal dengan efisiensi lebih tinggi
Efisiensi	Sangat tidak efisien untuk jumlah barang besar	Efisien, tidak menghitung hasil yang sama berulang kali
Memori	Tidak memerlukan tabel, hanya perhitungan langsung	Memerlukan tabel DP untuk menyimpan hasil sementara

Kesimpulan:

- Optimalitas dengan DP: Kombinasi barang yang dipilih dalam DP adalah yang optimal karena DP menyelesaikan masalah dengan memanfaatkan hasil perhitungan sub-masalah sebelumnya, memastikan pemilihan barang yang memaksimalkan keuntungan tanpa melebihi kapasitas.
- Brute Force: Walaupun brute force dapat memberikan solusi yang benar, pendekatan ini sangat tidak efisien untuk jumlah barang yang banyak. Setiap kombinasi dihitung ulang, yang membuatnya sangat lambat.

Dengan Dynamic Programming, kita dapat menyelesaikan masalah 0/1 Knapsack dengan jauh lebih cepat dan efisien, bahkan untuk masalah besar, sementara brute force hanya cocok untuk masalah kecil dengan jumlah barang yang terbatas.